RutaSmart — Pruebas de Seguridad

OWASP MASVS · análisis estático · vulnerabilidades y re-pruebas

Sarah Isabella Peña Cortes

Docente: Ing. Jairo Armando Salcedo Aranda

Universidad de San Buenaventura, Bogotá · Optativa II: Desarrollo de Aplicaciones para Dispositivos Móviles · Noviembre de 2026

Tabla de contenido

Información del documento 3

1. Objetivo y alcance 4

2. Herramientas 5

3. Controles implementados desde el diseño 6

4. Análisis estático (resultado real) 7

5. Hallazgos, riesgo y acciones correctivas 8

6. Autenticación y control de acceso 9

    6.1 Pruebas de autorización en Firestore (Rules Playground) 9

7. Almacenamiento local 11

8. Comunicaciones 12

9. Prácticas con Kali Linux y BadStore (laboratorio) 13

10. Re-pruebas 14

11. Conclusión 15

Información del documento

Campo	Valor
Proyecto	RutaSmart — Asistente inteligente para selección de alternativas de transporte
Documento	RS-SEC-001 · Pruebas de Seguridad de la Aplicación Móvil
Estándar de referencia	OWASP MASVS v2 / OWASP MASTG
Versión	1.0
Autora	Sarah Isabella Peña Cortes (sipenac@academia.usbbog.edu.co)
Asignatura	Optativa II: Desarrollo de Aplicaciones para Dispositivos Móviles
Docente	Ing. Jairo Armando Salcedo Aranda
Institución	Universidad de San Buenaventura, Bogotá
Fecha	Noviembre de 2026

1. Objetivo y alcance

Este documento registra las pruebas de seguridad de RutaSmart (paquete com.sarahpena.rutasmart) según **OWASP MASVS** y la guía **OWASP MASTG**. Las pruebas se hacen solo sobre la aplicación propia, en emulador o dispositivo propio. Las prácticas con Kali Linux se hacen contra **BadStore**, una tienda web vulnerable a propósito para laboratorio.

2. Herramientas

Herramienta	Tipo	Uso
mobsfscan v1.1.0 (reglas de MobSF)	Análisis estático (SAST)	Código Kotlin y AndroidManifest.xml contra reglas OWASP.
detect-secrets v1.5.0	Búsqueda de secretos	Claves, tokens y contraseñas en todo el repositorio.
JUnit (65 pruebas)	Pruebas unitarias	Validación de entradas, controles del asistente, excepciones.
Android Lint	Análisis estático	Code → Inspect Code / ./gradlew lint (también en Azure Pipelines).
Firebase Rules Playground	Pruebas de autorización	Acceso a datos de otros usuarios.
Kali Linux: Burp Suite / OWASP ZAP	Proxy	Verificar que todo el tráfico es HTTPS.
Kali Linux: nmap, nikto, sqlmap	Laboratorio	Prácticas contra BadStore.
adb	Inspección	Almacenamiento local, permisos y respaldos.

3. Controles implementados desde el diseño

Control	Implementación	MASVS
Respaldo deshabilitado	allowBackup="false", fullBackupContent="false", data_extraction_rules.xml excluye todo.	STORAGE
Sin tráfico en claro	usesCleartextTraffic="false"; Firebase y Gemini por HTTPS.	NETWORK
Protección contra secuestro de tareas (StrandHogg)	taskAffinity="" y launchMode="singleInstance".	PLATFORM
Clave de IA fuera del código	local.properties (en .gitignore) → BuildConfig; viaja en encabezado, nunca en la URL.	STORAGE / CODE
Reglas de Firestore por usuario	Solo el dueño (request.auth.uid == uid) lee y escribe; se validan tipos y rangos.	AUTH
Validación de entradas	ValidadorCuenta, BuscarLugarUseCase, CriteriosUsuario (0–5), AiInputGuard (500 car., manipulación).	CODE
Log solo en depuración	La Bitácora se conecta a Logcat únicamente con BuildConfig.DEBUG.	STORAGE
Permisos mínimos	INTERNET y ACCESS_NETWORK_STATE (permisos normales); sin ubicación, cámara ni almacenamiento.	PLATFORM / PRIVACY
Límite de uso de la IA	10 mensajes por minuto.	CODE

4. Análisis estático (resultado real)

Ejecutado sobre app/. El reporte completo está en mobsfscan_resultado.json.

Resultado: 0 hallazgos de severidad alta o media; 7 informativos.

Regla	Severidad	MASVS	Tratamiento
android_certificate_transparency	Informativa	MSTG-NETWORK-4	Ver tabla de hallazgos
android_kotlin_logging	Informativa	MSTG-STORAGE-3	Ver tabla de hallazgos
android_prevent_screenshot	Informativa	MSTG-STORAGE-9	Ver tabla de hallazgos
android_root_detection	Informativa	MSTG-RESILIENCE-1	Ver tabla de hallazgos
android_safetynet	Informativa	MSTG-RESILIENCE-1	Ver tabla de hallazgos
android_ssl_pinning	Informativa	MSTG-NETWORK-4	Ver tabla de hallazgos
android_tapjacking	Informativa	MSTG-PLATFORM-9	Ver tabla de hallazgos

detect-secrets: **0 secretos** encontrados en el repositorio (detect_secrets_resultado.json).

Comandos:

```
pip install mobsfscan detect-secrets
mobsfscan app/
detect-secrets scan --all-files
```

5. Hallazgos, riesgo y acciones correctivas

ID	Hallazgo	MASVS	Riesgo	Acción correctiva	Estado tras re-prueba
V-01	La bitácora escribía en Logcat también en la versión de producción (android_kotlin_logging).	STORAGE	Bajo	Conectar el log solo con BuildConfig.DEBUG; no se registran contraseñas ni claves.	✅ Corregido (la regla informa el uso de Log, que queda limitado a depuración)
V-02	La clave de Gemini queda dentro del APK compilado.	STORAGE	Medio	Restringir la clave en Google Cloud a paquete + SHA-1 y solo a la API de Gemini; a futuro, backend propio.	⚠️ Mitigado; acción operativa pendiente
V-03	El asistente podría recibir intentos de manipulación (prompt injection) o mensajes enormes.	CODE	Medio	AiInputGuard, límites de longitud y por minuto; instrucciones del sistema restringidas.	✅ 13 pruebas AiCoreTest
V-04	Sin certificate pinning ni Certificate Transparency.	NETWORK	Bajo	Aceptado: TLS del sistema; el pinning con Google puede romper la app al rotar certificados.	Aceptado
V-05	Sin detección de root ni Play Integrity.	RESILIENCE	Bajo	Aceptado: app académica sin pagos ni datos críticos.	Aceptado
V-06	Sin bloqueo de capturas de pantalla ni de tapjacking.	PLATFORM	Bajo	Aceptado: no se muestran datos sensibles; las acciones destructivas piden confirmación.	Aceptado

6. Autenticación y control de acceso

Verificación	Resultado esperado	Resultado
Registro con correo inválido o clave < 6	Mensaje de error, no se llama a Firebase	✅ Prueba automática
Login con clave incorrecta	«Correo o contraseña incorrectos» (no revela si el correo existe)	☐
Cerrar sesión	Vuelve a Login; los datos del usuario anterior no se muestran	☐
Historial por usuario	Cada usuario solo ve sus consultas (consultas por usuarioId en Room y por uid en Firestore)	☐

6.1 Pruebas de autorización en Firestore (Rules Playground)

En Firebase Console → Firestore → Reglas → **Área de pruebas**, autenticado con uid = A:

ID	Operación simulada	Esperado	Resultado
AZ-01	get /usuarios/A	Permitido	☐
AZ-02	get /usuarios/B	**Denegado**	☐
AZ-03	get /usuarios/B/consultas/x	**Denegado**	☐
AZ-04	create /usuarios/A/consultas/x con puntaje 150	**Denegado** (rango)	☐
AZ-05	create /usuarios/A/consultas/x válido	Permitido	☐
AZ-06	get /usuarios/A sin autenticación	**Denegado**	☐
AZ-07	write /otra_coleccion/x	**Denegado**	☐
AZ-08	create /admins/A (volverse admin)	**Denegado**	☐
AZ-08b	Registrar de nuevo admin@rutasmart.com	**Denegado** por Auth: «Ya existe una cuenta con ese correo»	☐
AZ-09	write /config/servicio siendo usuario normal	**Denegado**	☐
AZ-10	write /config/servicio siendo admin, aviso de 300 caracteres	**Denegado** (tamaño)	☐
AZ-11	write /config/servicio siendo admin, datos válidos	Permitido	☐
AZ-12	get /config/servicio sin autenticación	Permitido (es público)	☐
AZ-13	list grupo «consultas» siendo usuario normal / admin	**Denegado** / Permitido	☐

Control de acceso por rol. El botón 🛠️ solo es una comodidad visual: aunque alguien modificara la app, Firestore rechaza toda escritura en `config/servicio` y toda lectura del grupo `consultas` si quien lo pide no inició sesión con `admin@rutasmart.com` ni tiene documento en `admins`.

7. Almacenamiento local

Dato	Dónde	¿Sensible?
Historial (nombres de lugares, puntaje, costo)	Room: rutasmart.db (almacenamiento privado de la app)	Bajo
Preferencias (pesos y medios)	Room	No
Sesión de Firebase	Almacenamiento privado gestionado por el SDK de Firebase	Sí, protegido por el sistema
Contraseñas	Nunca se guardan	—
Ubicación GPS	No se usa	—

Verificación con adb (en compilación de depuración):

```
adb shell run-as com.sarahpena.rutasmart ls databases
adb backup com.sarahpena.rutasmart     # debe quedar vacío por allowBackup=false
adb shell dumpsys package com.sarahpena.rutasmart | grep permission
```

8. Comunicaciones

Servicio	Host	Protocolo
Firebase Auth / Firestore	*.googleapis.com (SDK)	HTTPS
Google Gemini	generativelanguage.googleapis.com	HTTPS
Google Maps (externo)	www.google.com/maps	HTTPS (Intent)

Prueba con Burp Suite (Kali):

1. Configurar el proxy del emulador hacia Burp.
2. Sin instalar el certificado de Burp, las conexiones deben **fallar**.
3. Ninguna petición debe aparecer en HTTP plano.

9. Prácticas con Kali Linux y BadStore (laboratorio)

BadStore se ejecuta en una máquina virtual aislada (red solo-anfitrión) junto a Kali Linux. Cada hallazgo se compara con el control de RutaSmart.

ID	Prueba en BadStore	Hallazgo esperado	Control en RutaSmart	Evidencia
K-01	nmap -sV <ip>	Servicios expuestos (HTTP, MySQL)	No expone servicios; solo consume APIs por HTTPS	☐
K-02	nikto -h http://<ip>	HTTP sin TLS, cabeceras inseguras	usesCleartextTraffic=false	☐
K-03	Inyección SQL en login / sqlmap	Acceso sin contraseña	Firebase Auth; Room con consultas parametrizadas (@Query con :parámetros)	☐
K-04	XSS en el libro de visitas	Script ejecutado	Compose muestra texto plano; AiInputGuard filtra el chat	☐
K-05	Manipular parámetros con Burp	Datos alterados aceptados	Reglas de Firestore validan tipos y rangos	☐

10. Re-pruebas

Hallazgo	Re-prueba	Resultado
V-01	Revisión de RutaSmartApp.kt + mobsfscan	✅ Log solo con BuildConfig.DEBUG
V-03	AiCoreTest (13 pruebas)	✅ 13/13
Secretos	detect-secrets	✅ 0 hallazgos
General	Suite completa JUnit	✅ 65/65

11. Conclusión

- El análisis estático no encontró vulnerabilidades altas ni medias, y no hay secretos en el repositorio.
- Los controles de respaldo, tráfico, secuestro de tareas, reglas por usuario y validación de entradas se incorporaron desde el diseño.
- Quedan pendientes las acciones operativas: restringir la clave de Gemini y ejecutar las pruebas del Rules Playground y del laboratorio con capturas.